

# Relatório Técnico-Científico: Maranhão

**Daniel Teixeira, Antonio Coqueiro, Mendell Kalyl, Vitor Sena, Augusto Santos**

UNEX - Universidade de Excelência

Caixa Postal 45020-510 - Vitória da Conquista - BA - Brasil

teixeira.santos18@aluno.unex.edu.br, coqueiro.costa@aluno.unex.edu.br,  
mendell.melo@aluno.unex.edu.br, vitor.senal@aluno.unex.edu.br,  
augusto.oliveira4@aluno.unex.edu.br

**Resumo.** Este relatório apresenta a implementação e aplicação do algoritmo de ordenação **Merge Sort** no sistema FoodDelivery Maranhão, desenvolvido como parte do processo de refatoração e evolução do sistema de gerenciamento de pedidos. O trabalho teve como objetivo modularizar o código, converter as entidades para mapas (dicionários), organizar dados armazenados em arquivos JSON e aplicar um algoritmo de ordenação eficiente para exibir pedidos de forma estruturada. O Merge Sort foi escolhido por sua estabilidade e desempenho consistente, sendo utilizado para ordenar os pedidos com base em seus identificadores. São discutidos os fundamentos teóricos, a metodologia de implementação, os resultados obtidos e as considerações finais sobre desafios enfrentados e aprendizados.

**Abstract.** This technical report presents the implementation and application of the **Merge Sort** sorting algorithm in the FooDDelivery Maranhão system, developed as part of the refactoring and evolution process of a food delivery order management application. The goal was to modularize the system, convert entities into Python dictionaries, store data in JSON files, and apply an efficient sorting method to organize and display orders. Merge Sort was selected due to its stability and consistent performance, and it was used to sort orders based on their identifiers. This report discusses the theoretical foundation, implementation methodology, obtained results, and final considerations regarding the challenges and insights gained throughout the development.

## 1. Introdução

A organização eficiente de informações é um fator essencial para sistemas que lidam com grande volume de dados, especialmente no contexto de restaurantes e serviços de entrega, onde pedidos precisam ser registrados, atualizados, consultados e ordenados rapidamente. O sistema foi desenvolvido em Python e passou por um processo de refatoração que incluiu a modularização de suas funções, a conversão de itens e pedidos

para mapas (dicionários), e a adoção de arquivos JSON como forma de persistência de dados.

## 2. Fundamentação Teórica

A organização e manipulação eficiente de dados são pilares fundamentais dentro da Ciência da Computação. Para sistemas que operam com grande volume de informações como sistemas de gerenciamento de pedidos, torna-se essencial adotar algoritmos de ordenação adequados e estruturas de dados robustas, que garantam velocidade, consistência e integridade na recuperação de elementos.

### 2.1 Ordenação

A ordenação permite maior organização, eficiência em buscas, legibilidade e manipulação simplificada das informações. Em sistemas reais, como plataformas de delivery, a ordenação dos pedidos seja por número, horário ou status é indispensável para garantir fluxo contínuo de trabalho.

### 2.2 Algoritmo Merge Sort

O Merge Sort é um algoritmo de ordenação baseado na estratégia **Dividir e Conquistar**. Ele funciona dividindo recursivamente a lista original em partes menores até que cada sublista possua apenas um elemento. Em seguida, essas sublistas são reorganizadas e mescladas de forma ordenada até reconstruir a lista final.

- Estável
- Complexidade:
  - Melhor caso:  $O(n \log n)$
  - Caso médio:  $O(n \log n)$
  - Pior caso:  $O(n \log n)$

## 3. Metodologia

A implementação seguiu uma arquitetura modular, com funções específicas para cada parte do sistema e dados estruturados de forma padronizada. A seguir, descrevemos os principais passos metodológicos.

### 3.1 Refatoração e Modularização

As funcionalidades foram separadas em módulos distintos para facilitar manutenção e leitura. O sistema conta com menus organizados para itens, pedidos e consultas, que direcionam o usuário às funções corretas.

### 3.2 Representação das Entidades

Todos os itens e pedidos foram representados através de **mapas (dicionários)**, contendo campos como:

- itemID, itemNome, itemPreco, itemEstoque
- idPedido, pedidoStatus, pedidoItens, totalPedido, cupomAplicado

Os dados são persistidos em arquivos **JSON**:

- itens.json
- pedidos.json

O carregamento inicial é realizado por:

- carregarItens()
- carregarPedidos()

### 3.3 Implementação do Merge Sort no Sistema

O algoritmo foi implementado nas funções:

- mergeSortPedidos(lista, numero)
- merge(metade1, metade2, numero)

Onde:

- lista = lista de pedidos
- numero = chave de ordenação (no caso, "idPedido")

Antes de exibir quaisquer pedidos, o sistema os ordena:

- pedidos\_ordenados = mergeSortPedidos(pedidos, numero="idPedido")

### 3.4 Fluxo operacional do sistema

Os principais processos implementados foram: Cadastro e atualização de itens; Controle de estoque; Criação de pedidos com cálculo automático de subtotal; Aplicação de cupons (OFF5, OFF10, OFF15); Processamento: aceitar ou rejeitar pedidos

Atualização de status: FAZENDO, PRONTO, ESPERANDO ENTREGADOR, SAÍDA PARA ENTREGA, ENTREGUE

- Cancelamento de pedidos
- Exibição ordenada e filtrada

## 4. Resultados e Discussões

Após a implementação do Merge Sort, o sistema apresentou maior clareza e organização na visualização dos pedidos. Entre os principais pontos observados:

### 4.1 Comportamento do Merge Sort

O algoritmo ordenou corretamente todos os pedidos pelo campo "idPedido", mesmo quando a lista possuía grande quantidade de entradas. A ordenação foi aplicada toda vez que o usuário solicitava a exibição completa dos pedidos.

```
| Número do pedido: 1 | Status: FAZENDO
| Itens do Pedido:
| 4 x1 | Preço unitário: R$4.00 | Subtotal: R$4.00 |
| Descoto aplicado no pedido: 0.0 | Valor total do pedido: R$4.00
| Número do pedido: 2 | Status: PENDENTE
| Itens do Pedido:
| 4 x2 | Preço unitário: R$4.00 | Subtotal: R$8.00 |
| Descoto aplicado no pedido: 0.0 | Valor total do pedido: R$8.00
Pressione ENTER para continuar...
```

Figura 1. Exibição de pedidos ordenados com Merge Sort

### 4.2 Cadastro de Itens

O sistema permite adicionar novos produtos informando nome, descrição, preço e estoque. Os itens são armazenados em lista e gravados no arquivo itens.json.

```
[.=====.]
[.-.-. Menu de Pedidos - Sistema de Pedidos FooDDelivery .-.-.]
[.=====.]
[.-.-.          1 - Criar Pedido          .-.-.]
[.-.-.          2 - Processar Pedidos     .-.-.]
[.-.-.          3 - Atualizar Pedidos     .-.-.]
[.-.-.          4 - Cancelar Pedidos     .-.-.]
[.-.-.          0 - Menu Principal       .-.-.]
[.=====.]
Escolha uma Opção: |
```

Figura 2. Tela de cadastro

### 4.3 Criação de pedidos

O usuário seleciona itens cadastrados e suas quantidades. O sistema calcula o total, aplica cupom (se houver) e registra o pedido como “pendente” em pedidos.json.

```
[.=====.]
[.-.-.  Menu de Itens - Sistema de Pedidos FooDDelivery  .-.-.]
[.=====.]
[.-.-.          1 - Cadastrar Itens          .-.-.]
[.-.-.          2 - Consultar Itens          .-.-.]
[.-.-.          3 - Atualizar Itens          .-.-.]
[.-.-.          0 - Menu Principal          .-.-.]
[.=====.]
Escolha uma Opção (Apenas número): █
```

Figura 3. Tela de criação dos pedidos

### 4.4 Processamento de pedidos

Os pedidos pendentes são avaliados e podem ser aceitos ou rejeitados. Ao aceitar, o sistema confirma o pedido; ao rejeitar, apenas atualiza seu status.

```
| Número do Pedido a Processar: 1 | Itens do Pedido: |
| Item: 4 x1 | Preço: R$4.00 | Subtotal: R$4.00 |
| Valor Total: R$4.00 |
Aceitar Pedido? (S/N): █
```

Figura 4. Processamento de pedidos

### 4.5 Atualização de status

Pedidos aceitos podem avançar por etapas como “fazendo”, “pronto”, “saída para entrega” e “entregue”, permitindo acompanhar todo o fluxo até a finalização.

```
=====
| Status Disponíveis: |
| 1 - Em Preparação |
| 2 - Preparo Finalizado |
| 3 - Esperando Entregador |
| 4 - Saída para Entrega |
| 5 - Entregue |
=====
Escolha o novo status: 1
Pedido Atualizado com Sucesso!
Pressione ENTER para continuar...█
```

**Figura 5. Atualização de status**

## **5. Considerações Finais**

A implementação do algoritmo MergeSort proporcionou resultados consistentes e uma organização clara dos pedidos. O algoritmo mostrou-se adequado pela sua estabilidade e desempenho previsível, especialmente em cenários com grande quantidade de dados.

Principais desafios:

- Refatoração completa do código original
- Modularização das funções
- Integração da ordenação com listas de dicionários
- Atualização dos arquivos JSON em tempo real

Pontos interessantes:

- Melhor organização do fluxo do sistema
- Maior clareza na exibição dos pedidos

Gerenciamento de estágios do pedido Melhorias futuras:

- Implementar interface gráfica
- Refinar validações e tratamentos de erro

## **6. Referências**

Cormen, T. H.; Leiserson, C. E.; Rivest, R. L.; Stein, C. (2002) Algoritmos: Teoria e Prática. Elsevier.

Curso em Vídeo (2017) Curso Python #01 - Seja um Programador. Disponível em: <https://www.youtube.com/watch?v=S9uPNppGsGo>.

SBC (s.d.) Instruções para Autores – Modelo SBC. Disponível em: <http://www.sbc.org.br>.